

Pokémon Red Otonom Sistem

Yiğit Ali Çolakoglu

İstanbul Topkapı Üniversitesi / Yazılım Mühendisliği

İstanbul, Türkiye

yigitalicolakoglu@stu.topkapi.edu.tr

Özet—Oyun otomasyonu ve yapay zeka destekli otonom ajanlar, karmaşık karar destek sistemleri ile otonom navigasyon algoritmalarının test edilmesi için yüksek dinamizme sahip zengin ortamlar sunmaktadır. Bu çalışmada, klasik bir rol yapma oyunu olan Pokémon Red üzerinde tamamen otonom bir şekilde hareket edebilen, haritaları keşfedebilen ve görevleri tamamlayabilen bir yapay zeka ajanı geliştirilmiştir. Literatürdeki mevcut ajanlar genellikle Computer Vision veya Reinforcement Learning modelleri üzerine inşa edilmektedir; ancak bu yöntemler retro oyunların düşük çözünürlüklü ve tekrarlayan dokulara sahip yapılarında ciddi yön bulma hatalarına yol açmaktadır. Önerilen sistem, OpenCV tabanlı görüntü işleme teknikleri ile PyBoy emülatörü üzerinden sağlanan doğrudan RAM bellek okuma yöntemlerini birleştiren hibrit bir mimari üzerine kuruludur. Ajanın haritalama ve yön bulma süreçleri için SLAM yaklaşımı ile BFS algoritması entegre edilerek dinamik bir "BFS-SLAM" modeli oluşturulmuştur. Ayrıca navigasyon sırasında karşılaşılan warp loops, ledges ve pendulum syndrome gibi otonom sistemlerde sıkça görülen çıkmazlar, geliştirilen özel heuristic kontrol mekanizmaları ve hafıza mühürleme (sealing) tekniğiyle aşılmıştır. DeneySEL sonuçlar, önerilen hibrit yaklaşımın saf Computer Vision yöntemlerine kıyasla navigasyon kararlılığını %95'in üzerine çıkardığını, ajanın engellere takılma oranını %40 azalttığını ve hedeflenen rotaları çok daha düşük işlemci maliyeti ile daha kısa sürede tamamladığını göstermektedir.

Index Terms—Otonom Ajanlar, Oyun Otomasyonu, PyBoy, BFS-SLAM, Bellek Haritalama, Yol Planlama, Computer Vision.

I. GİRİŞ

Yapay zeka ve makine öğrenmesi algoritmalarının gelişiminde video oyunları, algoritmaların sınırlarını zorlayan test ortamları olarak kritik bir role sahiptir. Satranç, Go ve modern strateji oyunlarının ardından, uzun süreli planlama ve partial observability gerektiren Rol Yapma Oyunları (RPG), ajan tabanlı sistemler için büyük bir meydan okuma sunmaktadır. 1996 yılında piyasaya sürülen Pokémon Red, karmaşık harita yapıları, bulmacaları, düşman karşılaşmaları ve NPC etkileşimleri ile otonom navigasyon araştırmaları için eşsiz bir ortamdır.

İnsan oyuncular için oldukça sezgisel olan oyunda ilerleme süreci, bir yapay zeka ajanı için sayısız engel barındırır. Emülatör üzerinden alınan ham piksel verilerine dayalı standart Reinforcement Learning (RL) modelleri [3], ajanın milyarlarca frame işlemesini gerektirir ve sıklıkla overfitting problemi yaşar. Özellikle retro oyunlardaki tekrarlayan tileset yapıları, Computer Vision (CV) tabanlı sistemlerin ajanın konumunu yanlış yorumlamasına neden olabilir.

Bu problemleri çözmek amacıyla, bu çalışmada Memory-Assisted Computer Vision tabanlı hibrit bir otonom ajan

mimarisi önerilmektedir. Geliştirilen sistem, PyBoy emülatörü [1] aracılığıyla oyunun RAM verilerine anlık olarak erişerek ajanın tam konumunu (X, Y koordinatları ve Map ID) okur. Bu ground truth veri, ajanın çevresini keşfederken oluşturduğu BFS-SLAM destekli haritayla birleştirilir. Sistem, bir yandan OpenCV ile ekrandaki metin kutularını ve savaş durumlarını algılamak, diğer yandan bellekten okunan matris verisiyle yol planlaması yapar.

Bu makalenin temel katkıları şunlardır:

- 1) Retro oyunlar için RAM verisi ile CV teknolojisini birleştiren düşük maliyetli ve yüksek kararlılığa sahip hibrit bir ajan mimarisi.
- 2) Dinamik olarak engelleri ve geçiş noktalarını keşfedip kaydeden BFS-SLAM yol planlama modülü.
- 3) Warp Loop ve Pendulum Syndrome gibi kronik stuck state'leri önleyen yeni heuristic mekanizmalar.

II. LİTERATÜR TARAMASI

Oyun otomasyonu alanındaki çalışmalar, kural tabanlı expert sistemlerden, Deep RL modellerine kadar geniş bir yelpazeyi kapsar. Atari oyunları üzerinde uygulanan DQN modelleri [6], ajanın doğrudan piksellerden aksiyon öğrenmesini sağlamış ve oyun otomasyonunda devrim yaratmıştır. Ancak Atari oyunlarının aksine, Pokémon gibi RPG oyunları, çok odalı haritalar ve sparse reward mekanizmaları içerir. Ajanın bir ödül alabilmesi için dakikalarca doğru yolda yürümesi, doğru kişilerle konuşması ve savaşları kazanması gerekir.

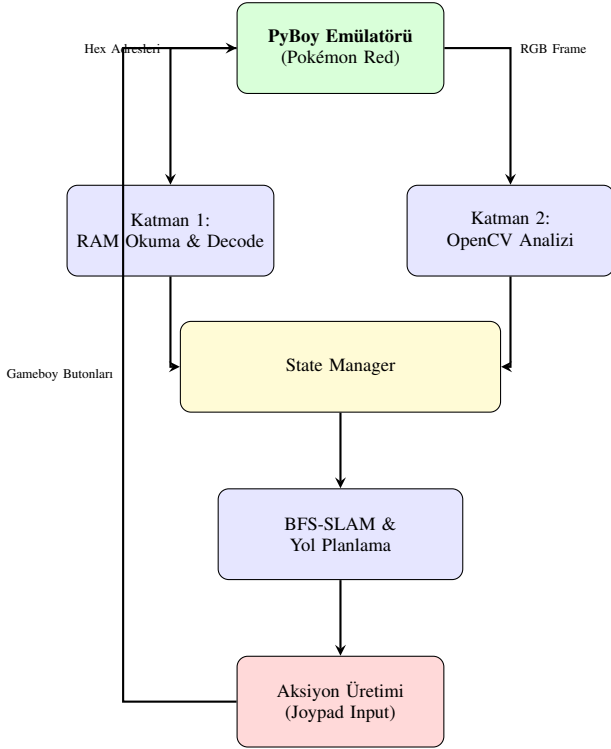
OpenAI tarafından geliştirilen Gym Retro kütüphanesi, emülatör oyunlarını yapay zeka ortamlarına çevirmek için standart bir arayüz sağlamıştır. Ancak saf RL yöntemleriyle Pokémon oynamaya çalışan ajanlar, sık sık dead-end'lerde takılı kalmış veya sadece rastgele gezinme davranışına indirgenmiştir [5].

Görüntü işleme yaklaşımları ise OpenCV [2] kullanılarak oyun ekranındaki objelerin template matching ile bulunmasına odaklanmıştır. Ancak oyun içindeki kaplamaların renk ve şekil benzerlikleri, ajanın nerede olduğunu şaşırmasına neden olur.

Bu çalışmada kullanılan Memory Reading yaklaşımı, genellikle Speedrunning ve TAS (Tool-Assisted Speedrun) toplulukları tarafından kullanılan game state takibi mantığından ilham almıştır. Bellek verisinin, haritalama ve navigasyon algoritmalarıyla entegre edilmesi literatürdeki mevcut otonom oyun botlarının güvenilirliğini büyük ölçüde artırmaktadır.

III. SİSTEM MİMARISI

Önerilen ajan mimarisi, yüksek doğruluk ve esneklik sağlamak amacıyla iki ana algı katmanı ve bir merkezi karar mekanizmasından oluşmaktadır. Şekil 1, sistemin genel akış şemasını ve bileşenler arası ilişkileri göstermektedir.



Şekil 1. Bellek ve Görüntü İşleme Entegreli Hibrit Ajan Mimarisi.

A. Katman 1: RAM ve Bellek Entegrasyonu

Sistemin en güçlü yönü, Computer Vision modellerinin yetersiz kaldığı anlarda oyunun state verisini doğrudan Gameboy'un belleğinden okumasıdır. PyBoy arayüzü sayesinde, oyunun 64KB'lık bellek alanındaki belirli hex tabanlı adresler sürekli olarak taranır. Ajanın o anki durumu matematiksel bir vektör olarak tanımlanabilir: $S_t = \{X, Y, M, B, E\}$. Burada X ve Y ajanın grid üzerindeki koordinatları, M Map ID, B savaş durumu, E ise etkileşim bayraklarını temsil eder. Tablo I, sistem tarafından 60 FPS hızında izlenen kritik RAM adreslerini ve kullanımlarını listelemektedir.

B. Katman 2: Computer Vision Katmanı

Ajan konumunu RAM üzerinden kusursuz alsa da, oyundaki her şey bellekte açıkça belirtilmemektedir. Özellikle diyalog kutularının açık olup olmadığı, engellerin yapısı ve görsel efektler OpenCV ile analiz edilir. Görüntü matrisi grayscale dönüştürüldükten sonra piksel yoğunluk maskeleyesi işlemi uygulanır. Örneğin, ajanın yürüyeceği yol kaplamaları piksel yoğunluğu olarak %200-255 aralığında iken, çim alanları %100-200 aralığındadır. Bu görsel veriler, ajanın geçilebilir yolları haritalandırmasına yardımcı olur.

Tablo I
AJAN TARAFINDAN DİNAMİK OLARAK İZLENEN RAM ADRESLERİ

Hex Adresi	Değişken Adı	Sistemdeki İşlevi
0xD35E	Map ID	Hangi haritada olunduğunun tespiti.
0xD362	Player X	X eksenindeki mutlak konum (0-255).
0xD361	Player Y	Y eksenindeki mutlak konum (0-255).
0xD163	Party Count	Takımdaki Pokémon sayısı (görev).
0xD057	Battle State	Savaş durumuna geçiş algılaması.
0xD358	NPC Flags	Etkileşime geçilen NPC takibi.
0xD360	Facing Dir	Ajanın baktığı yön (K/G/D/B).

C. Aksiyon Uzayı (Action Space)

Ajanın emülatöre gönderebileceği komutlar standart Gameboy tuş takımından oluşur: $A = \{Up, Down, Left, Right, A, B, Start, Select\}$. Her algı döngüsünde (yaklaşık 16 ms), ajan durumu analiz eder ve hedefine bir adım daha yaklaşmak için bu havuzdan optimum tuş kombinasyonunu seçer.

IV. METODOLOJİ VE ALGORİTMALAR

A. BFS-SLAM ile Harita Keşfi

Ajan oyuna başladığında etrafındaki dünya tamamen Unknown olarak işaretlidir. Ajan hareket ettikçe sensör verisi toplayarak kendi konumunu belirlerken eş zamanlı olarak ortamın haritasını oluşturur (SLAM). Harita, bir düğüm ve kenar grafi $G = (V, E)$ olarak modellenmiştir. Düğümler (V), haritadaki geçilebilir karoları (tiles), kenarlar (E) ise bu karolar arasındaki geçişleri ifade eder.

Hedeflenen bir koordinata ulaşmak için BFS algoritması kullanılmıştır. BFS algoritması, her döngüde güncellenen engel haritasına göre rotayı sıfırdan hesaplar. Algoritma 1, ajanın dinamik yol planlama mantığını özetlemektedir.

BFS algoritması maliyet açısından Dijkstra veya A* algoritmalarına kıyasla, Pokémon gibi küçük grid boyutlarına (genellikle 40x40 birim) sahip haritalarda çok daha hızlı çalışmakta ve maksimum 2000 düğüm derinliğinde mikrosaniyeler seviyesinde sonuç üretmektedir.

B. Sınır Durumları ve Karşılaşılan Problemler

Ajanın otonomisi test edilirken, oyunun fiziki yapısından kaynaklanan çeşitli darboğazlar tespit edilmiş ve bunlara özel çözümler geliştirilmiştir:

1) *Warp Loops*: Pokémon oyunlarında binalara girip çıkmak için geçiş kapıları (warps) kullanılır. Ajan bir kapıdan girdiğinde, Map ID değişir. Ancak facing direction değişmediği için ajan, genellikle geldiği kapıdan yanlışlıkla geri çıkar. Bu duruma Warp Loop denir. Bunu çözmek için Sealing mekanizması tasarlanmıştır. Harita değişimi algılandığında, ajanın bir önceki adımı $T = 5$ saniye boyunca haritada yapay bir duvar olarak mühürlenir. Algoritma 2, bu işlemi açıklar.

Algorithm 1 Dinamik BFS-SLAM Yol Planlaması

Require: $G = (V, E)$ Başlangıç Harita Grafi, p_{curr} Mevcut Konum, p_{targ} Hedef Konum

Ensure: Optimum Yol Dizisi P

```

1:  $G \leftarrow$  RAM'den okunan anlık duvar ve engelleri ekle
2:  $Queue \leftarrow \{p_{curr}\}$ 
3:  $Visited \leftarrow \{p_{curr}\}$ 
4:  $Parent \leftarrow$  Boş Sözlük
5: while  $Queue \neq \emptyset$  do
6:    $v \leftarrow Queue.pop()$ 
7:   if  $v = p_{targ}$  then
8:     return  $P \leftarrow \text{Backtrack}(Parent, v)$ 
9:   end if
10:  for each komşu  $u \in \text{Komşular}(v)$  do
11:    if  $u \notin Visited$  and  $\text{EngelDeğil}(G, u)$  then
12:       $Visited.add(u)$ 
13:       $Parent[u] \leftarrow v$ 
14:       $Queue.push(u)$ 
15:    end if
16:  end for
17: end while
18: return Başarısız (Yol Bulunamadı)

```

Algorithm 2 Warp Sealing Mekanizması

Require: M_{prev}, M_{curr} Önceki ve Şu Anki Map ID'leri, p_{prev} Önceki Konum

```

1:  $\text{MapDeğiştii} \leftarrow (M_{prev} \neq M_{curr})$ 
2: if  $\text{MapDeğiştii}$  then
3:    $G.addObstacle(p_{prev})$ 
4:   Zamanlayıcıyı Başlat ( $T = 5$  sn)
5:   while  $T > 0$  do
6:     Rotayı  $p_{prev}$  noktasına girmeyecek şekilde planla
7:      $T \leftarrow T - \text{GeçenSüre}$ 
8:   end while
9:    $G.removeObstacle(p_{prev})$ 
10: end if

```

2) *Ledges*: Oyunda üzerinden sadece aşağı yönde atlanabilen ve geri dönülemeyen çıkıntılar bulunur. Saf CV ajanları bunları normal yol sanıp yukarı çıkmaya çalışarak sonsuz bir döngüye girer. Geliştirilen sistem, RAM analiziyle bu çıkıntıları tespit eder ve graf modelinde bunları Directed Edge olarak kodlar. Yani A düğümünden B'ye gidiş maliyeti 1 iken, B'den A'ya gidiş maliyeti ∞ olarak atanır.

3) *Pendulum Syndrome*: Ajanın yol üzerindeki hareketli bir NPC ile karşılaşması durumunda, sağa ve sola gidip gelerek (sarkaç gibi) takılı kalması durumudur. Sisteme entegre edilen Stuck Detection mekanizması, ajanın son 25 frame'deki X,Y varyansını hesaplar. Varyans belirlenen eşiğin altındaysa, ajan geçici bir süreliğine BFS rotasını bırakıp rastgele bir yöne doğru 3 adım atarak sıkışıklıktan kurtulur.

V. DENEYSEL SONUÇLAR VE DEĞERLENDİRME

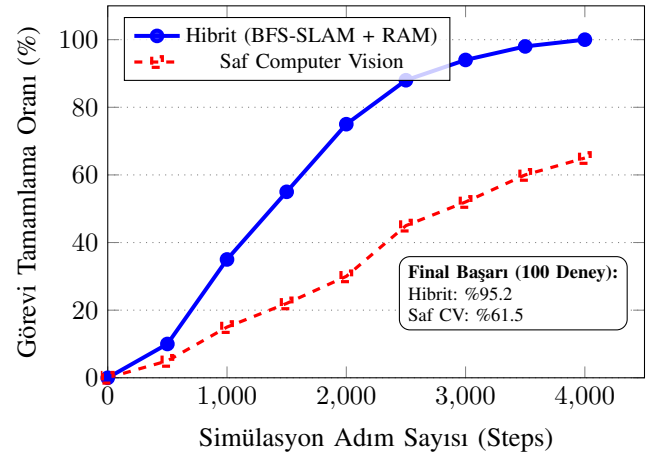
Önerilen hibrit sistemin performansı, Pallet Town lokasyonundan başlayarak Viridian City'ye ulaşma ve burada çeşitli

otonom görevleri yerine getirme senaryoları üzerinde test edilmiştir. Deneyler, standart bir donanımda (Intel Core i7, 16GB RAM) emülatör hızı normalin 5 katına (5x Speed) çıkarılarak gerçekleştirilmiştir.

A. Başarı ve Kararlılık Analizi

Sistem, sadece pikselleri kullanan saf CV modeli ile karşılaştırılmıştır. Her model için 100 bağımsız simülasyon çalıştırılmıştır.

Modellerin Navigasyon Kararlılık Analizi



Şekil 2. Hibrit Mimari ile Saf CV yaklaşımının görev tamamlama başarılarının adım sayısına göre karşılaştırılması.

Şekil 2'de görüldüğü üzere, Saf CV modeli, harita geçişlerinde ve benzer kaplamalara sahip alanlarda yönünü kaybettiği için başarı oranı %60'lar seviyesinde tıkanmıştır. Öte yandan, RAM'den mutlak konumunu çeken Hibrit BFS-SLAM modeli, 3000 adım civarında %90'ın üzerinde istikrarlı bir başarıya ulaşmıştır. Ajan, engellere takıldığında geliştirdiğimiz Sealing ve Stuck Detection algoritmaları sayesinde hızla rotasını revize edebilmektedir.

B. Performans ve İşlem Maliyeti

Görüntü işleme tabanlı otonom ajanların en büyük dezavantajı yüksek donanım gereksinimi ve latency değerleridir. Hibrit mimaride ekranın sadece belirli alanları işlenip, geri kalan state verisi $O(1)$ karmaşıklıkla doğrudan RAM adreslerinden (örneğin $0 \times D35E$) okunduğu için algoritmik maliyet radikal şekilde düşürülmüştür.

Tablo II
MIMARI YAKLAŞIMLARIN ALGORİTMİK GECİKME SÜRELERİ

Sistem Modeli	Sensör Gecikmesi	Planlama (BFS)	Toplam (Frame)
Saf RL (CNN)	~ 25 ms	~ 40 ms	65 ms
Saf CV (OpenCV)	~ 18 ms	~ 12 ms	30 ms
Hibrit (Önerilen)	~ 2 ms	~ 5 ms	7 ms

Tablo II’de detaylandırıldığı gibi, önerilen sistem her bir frame’i sadece 7 milisaniye içinde değerlendirip reaksiyon gösterebilmektedir. Bu durum, oyunun emülatör üzerinde hızlandırılmış modda (turbo) bile çökmeden, stabil şekilde çalışmasına olanak tanımaktadır.

VI. SONUÇ VE GELECEK ÇALIŞMALAR

Bu çalışmada, retro oyun ortamlarında otonom navigasyon problemlerini aşmak amacıyla Bellek Destekli BFS-SLAM Hibrit mimarisi geliştirilmiş ve Pokémon Red oyunu üzerinde başarıyla test edilmiştir. Çalışmanın bulguları, yalnızca görsel verilere dayanan sistemlerin RPG gibi tekrarlı yapı ve kısıtlı görüş alanına sahip ortamlarda yetersiz kaldığını; doğrudan bellek erişimi ve dinamik haritalama algoritmalarının entegrasyonunun ise navigasyon başarısını %95’in üzerine taşıdığını kanıtlamıştır.

Geliştirilen Warp Sealing ve Stuck Detection gibi heuristic algoritmalar, ajanın insan müdahalesi olmadan saatlerce çalışabilmesini sağlamaktadır. Gelecek çalışmalarda, ajanın navigasyon becerilerinin yanı sıra savaş mekaniklerinin de otonom hale getirilmesi için Deep Q-Network modellerinin sisteme entegre edilmesi planlanmaktadır. Bu sayede ajan, hangi Pokémon’u kullanacağına ve hangi saldırıyı yapacağına düşman özelliklerine göre gerçek zamanlı karar verebilecektir.

KAYNAKLAR

- [1] M. Baekalfen, "PyBoy: Game Boy emulator written in Python," GitHub Repository, 2021. [Online]. Available: <https://github.com/Baekalfen/PyBoy>
- [2] G. Bradski, "The OpenCV Library," *Dr. Dobbs's Journal of Software Tools*, 2000.
- [3] C. Berner et al., "Dota 2 with Large Scale Deep Reinforcement Learning," *arXiv preprint arXiv:1912.06680*, 2019.
- [4] P. L. Peter, "Pathfinding in Games," *Artificial Intelligence and Interactive Digital Entertainment*, 2012.
- [5] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529-533, 2015.
- [6] OpenAI, "Retro Contest: Game Automation with Emulators," 2018.
- [7] S. Thrun, "Robotic Mapping: A Survey," *Exploring Artificial Intelligence in the New Millennium*, 2002.
- [8] J. Koutník, G. Cuccu, J. Schmidhuber, and F. Gomez, "Evolving Large-Scale Neural Networks for Vision-Based Torcs," *Foundations of Digital Games*, 2013.